

Cost-Efficient Retrieval-Augmented Generation using Adaptive Vector Caching

Artificial Intelligence Problem Statement

Hardly Human — IIT Dharwad

Mentor: Prof. B. N. Bharath

1. Problem Statement

Retrieval-Augmented Generation (RAG) systems rely on vector databases that store embeddings of large document collections for semantic search. However, storing embeddings for very large corpora can be expensive in terms of memory and infrastructure. In many real-world applications, user queries tend to concentrate around a small subset of frequently accessed documents.

This project aims to design and implement a two-tier RAG architecture with an adaptive vector cache. Instead of embedding the entire document corpus, the system will maintain a small *hot* vector database containing embeddings of frequently accessed documents, while the remaining documents are stored in raw text form in cold storage.

When a query arrives, the system first checks the vector cache. If no relevant embedding is found, it performs text search in the cold storage, retrieves the relevant document and decides whether to store it in the cache.

The project will explore cache admission policies, cost-performance trade-offs, and retrieval accuracy, with the goal of reducing vector storage costs while maintaining RAG performance.

2. Provided Resources & Required Setup

Teams will receive the following resources:

- A collection of relevant databases.
- Access to a base Large Language Model through an API or open-source model
- Tools for document indexing and retrieval

Recommended development setup:

Component	Requirement
Programming Language	Python
Frameworks	PyTorch / TensorFlow (optional)
Libraries	Vector databases, RAG frameworks, retrieval libraries
Evaluation Tools	Retrieval accuracy, latency, storage cost metrics

Training datasets or documents used during development will not necessarily be used during final evaluation.

3. Tasks to be Implemented

Teams must design and implement a RAG pipeline with adaptive vector caching.

3.1 Two-Tier Document Storage

Implement a system with two levels of storage:

- **Hot Storage:** A vector database containing embeddings of frequently accessed documents.
- **Cold Storage:** A repository containing raw document text without precomputed embeddings.

3.2 Query Processing Pipeline

For each incoming query:

- Search the vector cache for relevant embeddings.
- If no relevant embedding is found, perform text search in cold storage.
- Generate embeddings for retrieved documents on demand.
- Decide whether the embedding should be stored in the vector cache.

3.3 Cache Admission Policies

Evaluate different strategies for deciding which documents remain in the cache, such as:

- Least Recently Used (LRU)
- Least Frequently Used (LFU)
- Learned or adaptive cache policies

3.4 LLM Integration

Use retrieved documents as context for a Large Language Model to generate responses for user queries.

3.5 Application Scenario

The system will operate over IIT Dharwad documents (with an access permission from the administration) or equivalent open access documents.

Users should be able to ask natural language questions about institute policies and documents.

4. Expected Outcomes

Teams are expected to implement:

- A caching-based Retrieval-Augmented Generation pipeline
- Evaluation of cache policies such as LRU, LFU, or learned policies
- Analysis of the trade-off between embedding cost and storage cost
- Experimental results on a realistic document corpus

5. Eligibility Requirements

Teams applying for this project should satisfy the following:

- Teams may consist of **1–4 students**.
- At least **one member** should have a **Computer Science or Mathematics and Computing background**.
- Teams should have a **good understanding of calculus**.
- Good programming skills are required (at least one language: **C, C++, or Python**).
- If at least one team member has completed a project related to **Machine Learning**, it will be an added advantage.
- Most importantly, students should be **motivated and hardworking**.

6. Selection Process

If the number of applying teams exceeds the available slots, teams will be **shortlisted based on a small evaluation conducted by Prof. B. N. Bharath**.

7. Final Submission Requirements

Each team must submit the following.

7.1 Source Code

Complete implementation of the adaptive RAG system including retrieval, caching, and LLM integration.

7.2 Demonstration Video (Maximum 3 minutes)

The video should demonstrate:

- Query input to the system
- Retrieval from vector cache and cold storage
- Generation of embeddings on demand
- Response generation using the LLM

7.3 Technical Document (Maximum 6 pages)

The report must include:

- System architecture
- Cache design and admission policies
- Retrieval and embedding pipeline
- Experimental setup and evaluation
- Analysis of cost-performance trade-offs

8. Evaluation

Submissions will be evaluated based on:

- Retrieval accuracy
- Storage efficiency
- System design and implementation
- Experimental analysis
- Demonstration of cost-efficient RAG performance

Additional dates and resources will be announced later.